Summary

Polymorphism

What is Polymorphism

From Greek "having many forms."

Main idea: We can treat related objects as the same kind of object in client code. At runtime, each object performs the correct task according to its actual class

Polymorphism ≠ Inheritance ⇨ Reusing code

↳ How use hierarchies to write flexible code

Pillars of Object-Oriented Programming

1. Encapsulation: bundling data and methods, hiding internal state
2. Inheritance (Specialization): creating subclasses that extend a superclass
3. Polymorphism (Abstraction): treating objects of different subclasses uniformly through a common interface

Three Steps

- Establish a common interface
- Implement the interface in each subclass
- Depend on the superclass in client code

4. Trade-off: Compile Time vs Runtime

Advantage: Client code is shorter, cleaner, and easier to maintain

Disadvantage: We won't know exactly what the client code will do until runtime, because it depends on the actual runtime class of the object

Conclusion: We trade one complexity (verbose if/else) for another (runtime decision-making)

Escaneado con CamScanner

How affect Extensibility

Polimorphism generally improves extensibility because any new behavior can be added by creating a new subclasses and the old code remains unchanged. This is the Open/Closed Principle in action. open For extension, closed for modification

Abstraction

Considering only the relevant parts of an object and ignoring the Others

Pawn: Include: location, get Possible Pawn Moves

Ignore: Shape, weight, creator

Class Activity

Two sistem were compared

Uber Taxi Simulator cars, trucks, bikes, people, pick-ups, user-controlled, grid terrain, vehicles

Dinosaur Game: dinos on a 2D grid, pickin up/throwing items, growing overtime, slowing when hit, power-ups

Similarities

Moving on a 2D grid

Picking up items

User-controller objects

Dissimilarities

Specific content

Abstraction process

Find similarities

Create a layer of abstraction that contains those similarities

Delegate specific differences to concrete subclasses

cannot stand on its own

Escaneado con CamScanner

in Java - specifics

Farm Animal.add(chicken);

chickens.add(chicken);

- A subclass object can be added to a superclass list
- A superclass object cannot be added to a subclass list without an explicit cast

Methods

Overriding A new definition of an existing method in a subclass (inheritance), Marked with @override

Overloading A new definition of a method in the same class, with the same name but different parameters

Key Takeaways

- Lests you write code that works on the superclass level while behaving correctly For each subclass at runtime,
- It eliminates long if/else + instance of chains.
- makes code extensible; new subclasses can be added without touching client code
- It relies on abstraction; Focusing on shared, relevant characteristics and hidding the rest
- The cost is that some decisions are deferred to runtime rather than resolved at compile time,

Escaneado con CamScanner

Fundation of user interface Design

Core Principle

"Keep the Simple Simple"

Users or Usage? ... Humans

Reflective questions about how we design for people

Design should be grounded in real human behavior not jvs what user says

Complexity

- What makes a system complex?
- As a system increases in complexity, should be the UI also increases in complexity? (No) the UI shield users from complexity.)
- What makes a UI complex?
- Essential vs. Accidental Complexity
  - Essential is inherent to the problem itself
  - Accidental comes from poor design, tools, or implementation.
- Goal: Handle essential complexity of the system and minimize accidental complexity.
- How can we keep simple thing simple?

4 Laws of Interface Design

1st Law: A computer shall not harm your work or, through inaction, allow your work to come to harm

2nd law: A computer shall not waste your time or require you to do more work than is strictly necessary

3rd Law: An interface is humane if it is responsive to human needs and considerate of human frailties

4th Law: The user should set the pace of interaction.

Escaneado con CamScanner

Sampling of Design Methodologies

Human-Centered Design

- Ensuring the interface is in accordance with human physicological facts

User-Centered Design

- Ensuring designs are driven by an accurate understanding of users, their needs, desires, and limitations

Usage-Centered Design

- Ensuring designs are based on users intention and usage patterns, using abstraction prototypes

6. Related Methodologies

- Goal-Directed Design (Cooper & Reimann, 2023)
- Usability Engineering (Nielsen 1993)
- Interaction Design (Preece, Rogers & Sharp, 2002)

7. Ineffective Methodologies for UI Development

The following approaches are NOT recommended for UI design.

- Database-or data-structure-driven design - letting the data model dictate the interface
- Development-environment or programming-language-driven design - letting the tools dictate the interface
- Platform or framework-driven design - letting the technology dictate the interface.

Why? Because they focus on technical concerns instead of human needs, leading to poor user experiences

Key Takeaways

- UI design must start from human needs, not from data structures, frameworks, or platforms.

Escaneado con CamScanner

- Complexity should be managed; handel essential complexity, minimize accidental complexity.
- Four laws guide humane interface design: don't harm work, don't waste time, be responsive and considerate, and let the user set the pace.
- Human-, User-, and Usage-Center Design are the recommended approaches
- Ineffective approaches are those driven by technology (databases, frameworks, languages) rather than by user and usage

Abstraction

What is Abstraction

From a Process Perspective

- Abstraction is the act of bringing certain details to the forefront while suppressing all others.
- John Guttag "The essence of abstractions is preserving information that is relevant in a given context

Abstraction (verb) Creating an interface for componet that exposes necessary details while hiding others

Abstraction (noun) Description that leaves out unnecessory details, A class definition is an abstraction

Creating good abstractions is hard - developers sift through many details and determine the most salients concepts

Chickens on a farm

- In a Farm information system, relevant properties; id, breed, birth date, molting status,

Filtering these properties is abstraction (verb), the resulting chicken class is an abstraction (noun)

Abstraction vs Encapsulation

Abstraction defines what is exposed

Encapsulation hides the implementation

Abstract Classes and Interfaces in Java

abstract class, Cannot be instantied

abstract method, Must be implemented by derived classes

interface, A contract a class implements an interface, Interface can inherit from other interfaces

Escaneado con CamScanner

Common Problems

Leaky abstraction Other components rely on details not explicity stated in the abstraction. Often solved with good encapsulation.

Over-abstraction Too much is forgotten, insufficient control or inadequate access to information

Definition and Benefits

For each component, there is an explicit and clear declaration of its accesible features. The exposed features should be no more and no less than what other components need.

Benefits: improved understandability, testability, maintainability, reusability, and modularity

Adherence Criteria for Good Abstraction

Meaningful labels and identifiers - precise, not too broad / narrow, classes/variables = nouns, methods = verbs

Context-aware labels and identifiers - names should make sense with their context

Explict declaration of the full abstraction everything exposed must be documented or declared

Abstraction sufficiency (completeness) - user can do what they need to do with the object

Abstraction conciseness (non-redundancy) - no redundant mechanisms unless required for legacy or adaptors

Key Takeaways

- Abstraction = expose only what is relevant, hide the rest
- A class definition is an abstraction
- Good abstraction requires meaningful names, context awareness, explicit declarations, sufficiency, and conciseness

Escaneado con CamScanner

- Avoid leaky abstractions and over-abstraction.
- Proper abstraction improves understandability, testability, maintainability, reusability, and modularity
- In Java abstract classes cannot be instantiated, interface defines a contract that classes implement.

Escaneado con CamScanner